

Skill

Task 1 : To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```
vennela_07@VennelaPC:~$ mkdir signal_handlers
vennela_07@VennelaPC:~$ cd signal_handlers
vennela_07@VennelaPC:~/signal_handlers$ nano signal_handler.c
vennela_07@VennelaPC:~/signal_handlers$ gcc signal_handler.c -o signal_handler
vennela_07@VennelaPC:~/signal_handlers$ ./signal_handler
Signal Handler Program Started
Process ID: 10089

Registered Signals:
SIGINT  -> Ctrl+C
SIGTERM -> Termination signal

Waiting for signals...
Press Ctrl+C to test SIGINT.
^C
```

```
GNU nano 7.2 signal_handler.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

/* Signal handler function */
void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received (Ctrl+C).\n");
        printf("Interrupt handled safely.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received.\n");
        printf("Termination signal handled.\n");
    }
}

int main()
{
    /* Register signal handlers */
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    printf("Signal Handler Program Started\n");
    printf("Process ID: %d\n", getpid());
}
```

```
printf("SIGTERM -> Termination signal\n");

printf("\nWaiting for signals...\n");
printf("Press Ctrl+C to test SIGINT.\n");

while (1)
{
    sleep(1);
}

return 0;
}
```

Task 2 : To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
vennela_07@VennelaPC:~$ mkdir sigint_foreground
vennela_07@VennelaPC:~$ cd sigint_foreground
vennela_07@VennelaPC:~/sigint_foreground$ nano sigint_foreground.c
vennela_07@VennelaPC:~/sigint_foreground$ gcc sigint_foreground.c -o sigint_foreground
vennela_07@VennelaPC:~/sigint_foreground$ ./sigint_foreground
Simple Foreground Job Controller
Shell PID: 11826

1. Start Foreground Job
2. Check Job Status
3. Exit
Enter choice: 1
Foreground job started with PID 11839

Foreground job started.
Job PID: 11839
Job is running for 20 seconds...
Job running: 1 second(s)
Job running: 2 second(s)
Job running: 3 second(s)
Job running: 4 second(s)
Job running: 5 second(s)
Job running: 6 second(s)
Job running: 7 second(s)
Job running: 8 second(s)
Job running: 9 second(s)
Job running: 10 second(s)
Job running: 11 second(s)
Job running: 12 second(s)
Job running: 13 second(s)
Job running: 14 second(s)
Job running: 15 second(s)
Job running: 16 second(s)
Job running: 17 second(s)
Job running: 18 second(s)
Job running: 19 second(s)
Job running: 20 second(s)
Foreground job completed.
Job completed normally.
```

```
GNU nano 7.2 sigint
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

/* SIGINT handler for the shell */
void handle_sigint(int sig)
{
    printf("\nShell received SIGINT.\n");

    if (foreground_pid > 0)
    {
        printf("Forwarding SIGINT to foreground job (PID: %d)\n",
            foreground_pid);

        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("No foreground job is running.\n");
    }
}

int main()
{
    pid_t pid;
    int status;
```

```

signal(SIGINT, handle_sigint);

printf("Simple Foreground Job Controller\n");
printf("Shell PID: %d\n", getpid());

while (1)
{
    printf("\n1. Start Foreground Job\n");
    printf("2. Check Job Status\n");
    printf("3. Exit\n");
    printf("Enter choice: ");

    int choice;
    scanf("%d", &choice);

    if (choice == 1)
    {
        if (foreground_pid > 0)
        {
            printf("A foreground job is already running.\n");
            continue;
        }

        pid = fork();

        if (pid < 0)
        {
            perror("fork");
            continue;
        }

        if (pid == 0)
        {
            /* Child process */
            printf("\nForeground job started.\n");
            printf("Job PID: %d\n", getpid());
            printf("Job is running for 20 seconds...\n");

            for (int i = 1; i <= 20; i++)
            {
                printf("Job running: %d second(s)\n", i);
                sleep(1);
            }

            printf("Foreground job completed.\n");

            pid = fork();

            if (pid < 0)
            {
                perror("fork");
                continue;
            }

            if (pid == 0)
            {
                /* Child process */
                printf("\nForeground job started.\n");
                printf("Job PID: %d\n", getpid());
                printf("Job is running for 20 seconds...\n");

                for (int i = 1; i <= 20; i++)
                {
                    printf("Job running: %d second(s)\n", i);
                    sleep(1);
                }

                printf("Foreground job completed.\n");
                exit(0);
            }
            else
            {

```

```
GNU nano 7.2 sigint_foreground.c

foreground_pid = pid;

printf("Foreground job started with PID %d\n",
      foreground_pid);

waitpid(foreground_pid, &status, 0);

if (WIFSIGNALED(status))
{
    printf("Job terminated by signal %d.\n",
          WTERMSIG(status));
}
else if (WIFEXITED(status))
{
    printf("Job completed normally.\n");
}

foreground_pid = -1;
}
}
else if (choice == 2)
{
    if (foreground_pid > 0)
    {
        printf("Foreground Job PID: %d\n", foreground_pid);
        printf("Status: Running\n");
    }
    else
    {
        printf("No foreground job is running.\n");
    }
}
else if (choice == 3)
{
    printf("Shell exiting...\n");
    break;
}
else
{
    printf("Invalid choice.\n");
}
}

return 0;
}
```

Task 3 : To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
vennela_07@vennelaPC:~$ C
vennela_07@VennelaPC:~$ mkdir sigtstp_jobs
vennela_07@VennelaPC:~$ cd sigtstp_jobs
vennela_07@VennelaPC:~/sigtstp_jobs$ cd sigtstp_jobs
-bash: cd: sigtstp_jobs: No such file or directory
vennela_07@VennelaPC:~/sigtstp_jobs$ nano sigtstp_jobs.c
```

```
vennela_07@VennelaPC:~/sigtstp_jobs$ gcc sigtstp_jobs.c -o sigtstp_jobs
vennela_07@VennelaPC:~/sigtstp_jobs$ ./sigtstp_jobs
SIGTSTP Job Suspension System
Shell PID: 826

1. Start Job
2. Show Job Table
3. Resume Job
4. Exit
Enter choice: 1

Foreground job started.
Job ID: 1
PID: 848

Job started.
Job PID: 848
Job running: 1 second(s)
Job running: 2 second(s)
Job running: 3 second(s)
Job running: 4 second(s)
Job running: 5 second(s)
Job running: 6 second(s)
Job running: 7 second(s)
Job running: 8 second(s)
Job running: 9 second(s)
Job running: 10 second(s)
Job running: 11 second(s)
Job running: 12 second(s)
Job running: 13 second(s)
Job running: 14 second(s)
Job running: 15 second(s)
^Z
No foreground job is running.
Job running: 16 second(s)

SIGTSTP received.
```

4. Exit

Enter choice: 2

```
-----  
Job ID   PID      Status  
-----  
1        848      Stopped  
-----
```

1. Start Job
2. Show Job Table
3. Resume Job
4. Exit

Enter choice: 3

Resuming Job 1...

Job resumed successfully.

1. Start Job
2. Show Job Table
3. Resume Job
4. Exit

Enter choice: Job running: 17 second(s)

Job running: 18 second(s)

Job running: 19 second(s)

Job running: 20 second(s)

Job running: 21 second(s)

^Z

No foreground job is running.

Job running: 22 second(s)

```

GNU nano 7.2 sigtstp_jobs.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

struct Job
{
    int job_id;
    pid_t pid;
    char state[20];
};

struct Job job;

pid_t child_pid = -1;

/* SIGTSTP handler */
void handle_sigtstp(int sig)
{
    if (child_pid > 0)
    {
        printf("\nSIGTSTP received.\n");
        printf("Suspending foreground process PID: %d\n", child_pid);

        kill(child_pid, SIGSTOP);

        job.state[0] = '\0';
        snprintf(job.state, sizeof(job.state), "Stopped");
    }
    else
    {
        printf("\nNo foreground job is running.\n");
    }
}

void show_job()
{
    printf("\n-----\n");
    printf("Job ID   PID   Status\n");
    printf("-----\n");

    if (job.pid > 0)
    {
        printf("%-8d %-8d %s\n",
            job.job_id, job.pid, job.state);
    }
    else
    {
        printf("No jobs available.\n");
    }

    printf("-----\n");
}

void resume_job()
{
    if (job.pid > 0 &&
        strcmp(job.state, "Stopped") == 0)
    {
        printf("\nResuming Job %d...\n", job.job_id);

        kill(job.pid, SIGCONT);

        snprintf(job.state, sizeof(job.state), "Running");

        printf("Job resumed successfully.\n");
    }
    else
    {

```

```

        snprintf(job.state,
            sizeof(job.state),
            "Completed");

        printf("\nJob state updated: Completed\n");

        child_pid = -1;
    }
}

}
else if (choice == 2)
{
    show_job();
}
else if (choice == 3)
{
    resume_job();
}
else if (choice == 4)
{
    printf("Exiting...\n");
    break;
}
else
{
    printf("Invalid choice.\n");
}
}

return 0;
}

```

Task 4 : To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```
vennela_07@VennelaPC:~$ mkdir process_groups
vennela_07@VennelaPC:~$ mkdir process_groups
mkdir: cannot create directory 'process_groups': File exists
vennela_07@VennelaPC:~$ cd process_groups
vennela_07@VennelaPC:~/process_groups$ nano process_groups.c
```

```
vennela_07@VennelaPC:~/process_groups$ gcc process_groups.c -o process_groups
vennela_07@VennelaPC:~/process_groups$ ./process_groups
Shell PID : 2495
Shell PGID : 2495

Parent created process group.
Job PID : 2496
Job PGID : 2496

Child process created.
Child PID : 2496
Child PGID : 2496
Child waiting for signal...

Child is stopped.
Terminal control transferred to PGID 2496.
Sending SIGCONT to process group...
Child resumed.
Child process group is active.
Child completed.

[1]+  Stopped                  ./process_groups
```

```
GNU nano 7.2 process_groups.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    pid_t pid;
    pid_t shell_pgid;
    pid_t job_pgid;
    int status;

    /* Store shell process group */
    shell_pgid = getpgrp();

    printf("Shell PID : %d\n", getpid());
    printf("Shell PGID : %d\n", shell_pgid);

    /* Create a child */
    pid = fork();
```

```

{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    /*
     * Child becomes leader of a new process group.
     */
    setpgid(0, 0);

    job_pgid = getpgrp();

    printf("\nChild process created.\n");
    printf("Child PID : %d\n", getpid());
    printf("Child PGID : %d\n", job_pgid);

    printf("Child waiting for signal...\n");

    /*
     * Wait until the parent sends SIGCONT.
     */
    raise(SIGSTOP);

    printf("Child resumed.\n");
    printf("Child process group is active.\n");

    sleep(3);

    printf("Child completed.\n");

    exit(0);
}
else
{
    /*
     * Parent assigns child to its own process group.
     */
    setpgid(pid, pid);

    job_pgid = pid;

    printf("\nParent created process group.\n");
    printf("Job PID : %d\n", pid);
    printf("Job PGID : %d\n", job_pgid);

    /*
     * Wait until child stops.
     */
    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        printf("\nChild is stopped.\n");

        /*
         * Give terminal control to the child process group.
         */
        if (tcsetpgrp(STDIN_FILENO, job_pgid) == -1)
        {
            perror("tcsetpgrp");
        }
        else
        {
            printf("Terminal control transferred to PGID %d.\n",
                job_pgid);
        }

        /*
         * Send SIGCONT to the complete process group.
         */
        printf("Sending SIGCONT to process group...\n");

        kill(-job_pgid, SIGCONT);

        /*
         * Wait for the child to finish.
         */
        waitpid(pid, &status, 0);

        /*
         * Restore terminal control to the shell.

```

```

    /*
    printf("Sending SIGCONT to process group...\n");
    kill(-job_pgid, SIGCONT);

    /*
    * Wait for the child to finish.
    */
    waitpid(pid, &status, 0);

    /*
    * Restore terminal control to the shell.
    */
    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf("\nTerminal control restored to shell.\n");
    }

    if (WIFEXITED(status))
    {
        printf("Child exited normally.\n");
    }
    }
}

printf("\nProcess group demonstration completed.\n");
return 0;
}
|

```

